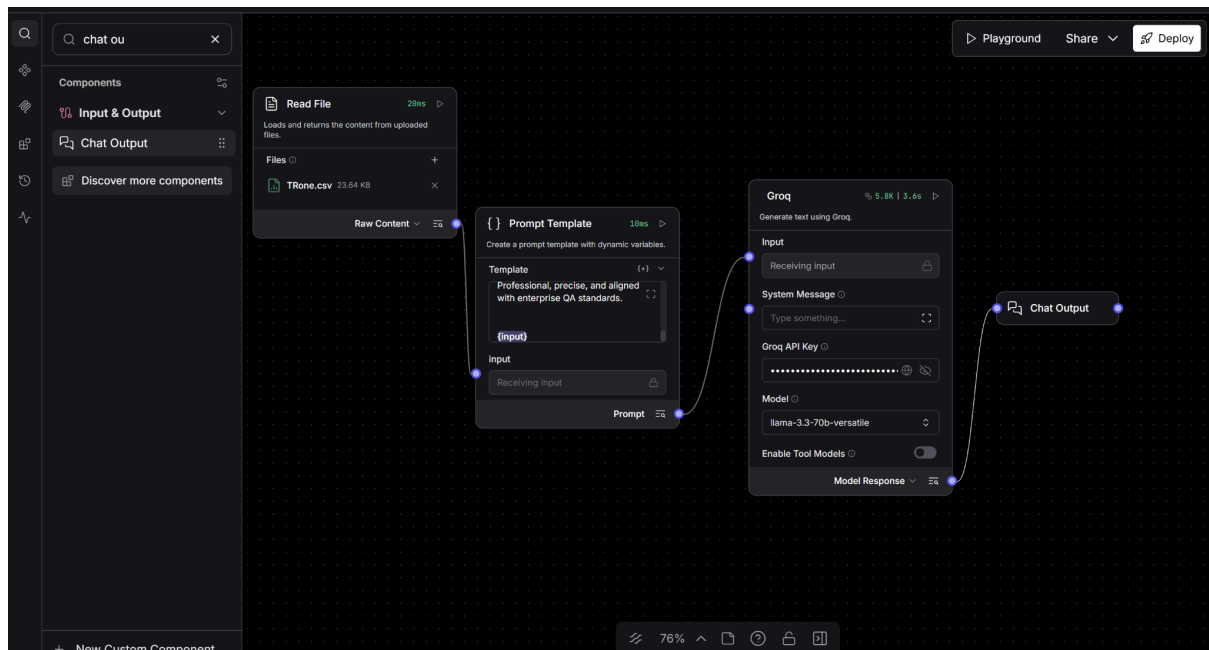


Lang Flow:



Prompt:

Strict Test Case Generator

I – Instruction

Validate the provided test cases for completeness, clarity, and adherence to industry best practices. Identify gaps, suggest improvements, Point out any missing steps and provide an enhanced version of each test case.

Validation Criteria:

Ensure coverage of:

- Positive test cases
- Negative test cases
- OWASP/security test scenarios
- Functional test cases
- End-to-End (E2E) scenarios

CRITICAL RULES (Non-Negotiable):

1. Each test case must be unique (no duplication).
2. All functionalities must be comprehensively covered.
3. Test cases must follow industry-standard structure (e.g., Test Case ID, Description, Preconditions, Test Steps, Expected Results, Priority, etc.).

C – Context

You are a Senior QA Test Lead in the Banking domain with extensive experience in reviewing and improving test cases.

O – Output Rules

Respond in JSON format

T – Tone

Professional, precise, and aligned with enterprise QA standards.

{input}

Response:

Test Case Validation and Enhancement

The provided test cases cover various aspects of the Yani AI system, including database connections, user interface, and security. However, there are some gaps and areas for improvement.

Positive Test Cases

- * The test cases cover basic functionality, such as adding a new database connection, displaying databases in the end-user interface, and selecting a specific database.
- * However, there are no test cases for positive scenarios, such as:
 - + Validating that the AI returns accurate results for a given query.
 - + Verifying that the system handles multiple database connections simultaneously.

Negative Test Cases

- * The test cases cover some negative scenarios, such as:
 - + Validating that the system blocks SQL injection attempts.
 - + Verifying that the system prevents unauthorized access to databases.
- * However, there are no test cases for other negative scenarios, such as:
 - + Validating that the system handles invalid or malformed queries.
 - + Verifying that the system prevents data corruption or loss due to database errors.

OWASP/Security Test Scenarios

- * The test cases cover some security-related scenarios, such as:
 - + Validating that the system blocks SQL injection attempts.
 - + Verifying that the system prevents unauthorized access to databases.
- * However, there are no test cases for other security-related scenarios, such as:

- + Validating that the system uses secure protocols for data transmission (e.g., HTTPS).
- + Verifying that the system implements proper input validation and sanitization.

Functional Test Cases

- * The test cases cover some functional scenarios, such as:
 - + Validating that the system displays databases in the end-user interface.
 - + Verifying that the system allows users to select a specific database.
- * However, there are no test cases for other functional scenarios, such as:
 - + Validating that the system provides accurate and relevant results for a given query.
 - + Verifying that the system handles multiple database connections simultaneously.

End-to-End (E2E) Scenarios

- * The test cases do not cover any end-to-end scenarios, such as:
 - + Validating that the system integrates with other components or services.
 - + Verifying that the system provides a seamless user experience from start to finish.

Enhanced Test Cases

Here is an example of how the test cases could be enhanced to cover the gaps and areas for improvement:

```
```json
[
 {
 "Test Case ID": "TC_153986_001",
 "Description": "Validate that an admin can successfully add a new database connection to the Yani AI system",
 "Preconditions": "Admin user is logged in",
 "Test Steps": [
 "Click on 'Add Database' button",
 "Enter valid database credentials",
 "Click on 'Save' button"
],
 "Expected Results": "Database connection is successfully added",
 "Priority": "High"
 },
 {
 "Test Case ID": "TC_153986_002",
 "Description": "Validate that all admin-configured databases are displayed in the end-user AI chat interface",
 "Preconditions": "Admin user has configured multiple databases",
 "Test Steps": [
 "Log in as end-user",
 "Click on 'Databases' tab",
 "Verify that all configured databases are displayed"
],
 },
]
```

```

 "Expected Results": "All configured databases are displayed",
 "Priority": "Medium"
 },
 {
 "Test Case ID": "TC_153986_003",
 "Description": "Validate that the AI returns accurate results for a given query",
 "Preconditions": "Database connection is established",
 "Test Steps": [
 "Enter a valid query",
 "Click on 'Submit' button",
 "Verify that accurate results are returned"
],
 "Expected Results": "Accurate results are returned",
 "Priority": "High"
 },
 {
 "Test Case ID": "TC_153986_004",
 "Description": "Validate that the system handles invalid or malformed queries",
 "Preconditions": "Database connection is established",
 "Test Steps": [
 "Enter an invalid or malformed query",
 "Click on 'Submit' button",
 "Verify that an error message is displayed"
],
 "Expected Results": "Error message is displayed",
 "Priority": "Medium"
 },
 {
 "Test Case ID": "TC_153986_005",
 "Description": "Validate that the system uses secure protocols for data transmission (e.g.,
HTTPS)",
 "Preconditions": "Database connection is established",
 "Test Steps": [
 "Verify that HTTPS protocol is used for data transmission"
],
 "Expected Results": "HTTPS protocol is used",
 "Priority": "High"
 }
]
...

```

Note that this is just an example and the actual test cases will depend on the specific requirements and functionality of the Yani AI system.

### ### JSON Output

Here is the enhanced test case data in JSON format:

```

```json
{

```

```

"testCases": [
  {
    "testCaseId": "TC_153986_001",
    "description": "Validate that an admin can successfully add a new database connection
to the Yani AI system",
    "preconditions": "Admin user is logged in",
    "testSteps": [
      "Click on 'Add Database' button",
      "Enter valid database credentials",
      "Click on 'Save' button"
    ],
    "expectedResults": "Database connection is successfully added",
    "priority": "High"
  },
  {
    "testCaseId": "TC_153986_002",
    "description": "Validate that all admin-configured databases are displayed in the end-user
AI chat interface",
    "preconditions": "Admin user has configured multiple databases",
    "testSteps": [
      "Log in as end-user",
      "Click on 'Databases' tab",
      "Verify that all configured databases are displayed"
    ],
    "expectedResults": "All configured databases are displayed",
    "priority": "Medium"
  },
  {
    "testCaseId": "TC_153986_003",
    "description": "Validate that the AI returns accurate results for a given query",
    "preconditions": "Database connection is established",
    "testSteps": [
      "Enter a valid query",
      "Click on 'Submit' button",
      "Verify that accurate results are returned"
    ],
    "expectedResults": "Accurate results are returned",
    "priority": "High"
  },
  {
    "testCaseId": "TC_153986_004",
    "description": "Validate that the system handles invalid or malformed queries",
    "preconditions": "Database connection is established",
    "testSteps": [
      "Enter an invalid or malformed query",
      "Click on 'Submit' button",
      "Verify that an error message is displayed"
    ],
  },

```

```
"expectedResults": "Error message is displayed",
"priority": "Medium"
},
{
  "testCaseId": "TC_153986_005",
  "description": "Validate that the system uses secure protocols for data transmission (e.g.,
HTTPS)",
  "preconditions": "Database connection is established",
  "testSteps": [
    "Verify that HTTPS protocol is used for data transmission"
  ],
  "expectedResults": "HTTPS protocol is used",
  "priority": "High"
}
]
}
```